

LUT_FINDER_TOOL

```
import numpy as np
from scipy.interpolate import interp1d
import warnings

class LUT_Finder:
    def __init__(self, npz_filepath):
        """Initializes the Agent and pre-computes all methodology
        matrices."""
        data = np.load(npz_filepath)
        self.L_vec = data['L_vector']
        self.V_vec = data['VX_vector']
        self.W_ref = 10e-6 # 10um width used during LUT generation

        # Dictionary holding all N x M matrices for fast lookup
        self.db = {'nmos': {}, 'pmos': {}}
        self._build_matrices(data)

    def _build_matrices(self, data):
        """Constructs the derived parameter tensors for g_m/I_D
        methodology."""
        for mos, pfx in zip(['nmos', 'pmos'], ['@m1', '@m2']):
            # Raw SPICE vectors (PMOS currents and caps are taken as
            absolute)

            id_raw = np.abs(data[f'{pfx}[id]'])
            gm_raw = data[f'{pfx}[gm]']
            gds_raw = data[f'{pfx}[gds]']
            cgs = np.abs(data[f'{pfx}[cgs]'])
            cgd = np.abs(data[f'{pfx}[cgd]'])
            cgb = np.abs(data[f'{pfx}[cgb]'])

            # Mask out sub-picoamp currents to prevent division by zero
            in cutoff

            id_clean = np.where(id_raw > 1e-12, id_raw, np.nan)
            gds_clean = np.where(gds_raw > 1e-12, gds_raw, np.nan)
```

```

# 1. Calculate Core Metrics
self.db[mos]['gm_ID'] = gm_raw / id_clean
self.db[mos]['gm_gds'] = gm_raw / gds_clean
self.db[mos]['gds_ID'] = gds_clean / id_clean
self.db[mos]['ID_W'] = id_clean / self.W_ref

#  $fT = gm / (2 * \pi * C_{gg})$  where  $C_{gg}$  is approx  $C_{gs} + C_{gd} +$ 
Cgb

cgg = cgs + cgd + cgb
self.db[mos]['fT'] = gm_raw / (2 * np.pi * cgg)

# VGS and Vth
self.db[mos]['VGS'] = self.V_vec # V_X maps to VGS or VSG
directly

self.db[mos]['Vth'] = data[f'{pfx}[vth]']

#
=====

# 2D Surface Search Functions (L is Unknown)
#
=====

def find_L_given_fT_gmgds(self, fT_Q, gm_gds_Q, mos_type):
    """1. Finds L given f_T and gm/gds (Intrinsic Gain)."""
    fT_mat = self.db[mos_type]['fT']
    gmgds_mat = self.db[mos_type]['gm_gds']

    # Calculate normalized 2D error surface to find the closest
intersection
    error = ((fT_mat - fT_Q) / fT_Q)**2 + ((gmgds_mat - gm_gds_Q) /
gm_gds_Q)**2
    idx_L, _ = np.unravel_index(np.nanargmin(error), error.shape)
    return self.L_vec[idx_L]

def find_L_given_gdsID_VGS(self, gds_ID_Q, VGS_Q, mos_type):
    """6. Finds L given gds/ID (lambda) and VGS/VSG."""
    idx_V = (np.abs(self.V_vec - VGS_Q)).argmin()
    gdsID_col = self.db[mos_type]['gds_ID'][:, idx_V]

```

```

        idx_L = (np.abs(gdsID_col - gds_ID_Q)).nanargmin()
        return self.L_vec[idx_L]

def find_L_given_gdsID_gmID(self, gds_ID_Q, gm_ID_Q, mos_type):
    """9. Finds L given gds/ID (lambda) and gm/ID."""
    gdsID_mat = self.db[mos_type]['gds_ID']
    gmID_mat = self.db[mos_type]['gm_ID']

    error = ((gdsID_mat - gds_ID_Q) / gds_ID_Q)**2 + ((gmID_mat -
gm_ID_Q) / gm_ID_Q)**2
    idx_L, _ = np.unravel_index(np.nanargmin(error), error.shape)
    return self.L_vec[idx_L]

#
=====

# 1D Vector Search Functions (L is Known)
#
=====

def _get_L_row(self, L_Q, mos_type, param_name):
    """Helper to extract a 1D parameter array for a specific
Length."""
    idx_L = (np.abs(self.L_vec - L_Q)).argmin()
    return self.db[mos_type][param_name][idx_L, :]

def _interpolate_1d(self, x_array, y_array, x_target,
param_name="Value"):
    """Helper to safely interpolate values, handling out-of-bounds
cleanly."""
    valid_idx = ~np.isnan(x_array) & ~np.isnan(y_array)
    x_clean, y_clean = x_array[valid_idx], y_array[valid_idx]

    # 1. Sort to prevent non-monotonic crashes
    sort_idx = np.argsort(x_clean)
    x_clean, y_clean = x_clean[sort_idx], y_clean[sort_idx]

    # 2. Remove duplicate x-values (SPICE numerical noise can create
duplicates)
    x_clean, unique_idx = np.unique(x_clean, return_index=True)

```

```

y_clean = y_clean[unique_idx]

# 3. Use bounds_error=False and fill_value=np.nan instead of
extrapolating
f = interp1d(x_clean, y_clean, kind='linear', bounds_error=False,
fill_value=np.nan)

result = float(f(x_target))

# 4. Warn the user if they request a physically impossible
operating point
if np.isnan(result):
    min_val, max_val = np.nanmin(x_clean), np.nanmax(x_clean)
    warnings.warn(
        f"Target {x_target} is out of bounds for this L. "
        f"Valid range is [{min_val:.3e}, {max_val:.3e}].
Returning NaN."
    )

return result

def find_ID_W_given_L_gmgds(self, L_Q, gm_gds_Q, mos_type):
    """2. Finds ID/W given L and gm/gds."""
    gmgds_row = self._get_L_row(L_Q, mos_type, 'gm_gds')
    idw_row = self._get_L_row(L_Q, mos_type, 'ID_W')
    return self._interpolate_1d(gmgds_row, idw_row, gm_gds_Q)

def find_gm_ID_given_L_gmgds(self, L_Q, gm_gds_Q, mos_type):
    """3. Finds gm/ID given L and gm/gds."""
    gmgds_row = self._get_L_row(L_Q, mos_type, 'gm_gds')
    gmid_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    return self._interpolate_1d(gmgds_row, gmid_row, gm_gds_Q)

def find_VGS_given_L_gmID(self, L_Q, gm_ID_Q, mos_type):
    """4. Finds VGS/VSG given L and gm/ID."""
    gmid_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    return self._interpolate_1d(gmid_row, self.V_vec, gm_ID_Q)

def find_Vth_given_L_gmID(self, L_Q, gm_ID_Q, mos_type):

```

```

        """5. Finds Vth given L and gm/ID."""
        gmid_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
        vth_row = self._get_L_row(L_Q, mos_type, 'Vth')
        return self._interpolate_1d(gmid_row, vth_row, gm_ID_Q)

def find_gm_ID_given_L_VGS(self, L_Q, VGS_Q, mos_type):
    """7. Finds gm/ID given L and VGS/VSG."""
    # Fix: Now correctly interpolates instead of snapping to grid
    gmid_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    return self._interpolate_1d(self.V_vec, gmid_row, VGS_Q)

def find_ID_W_given_L_gmID(self, L_Q, gm_ID_Q, mos_type):
    """8 & 10. Finds ID/W given L and gm/ID."""
    gmid_row = self._get_L_row(L_Q, mos_type, 'gm_ID')
    idw_row = self._get_L_row(L_Q, mos_type, 'ID_W')
    return self._interpolate_1d(gmid_row, idw_row, gm_ID_Q)

```